LABORATÓRIO TPSE I

UFC

Prática 01: Ambiente de Desenvolvendo e Primeiro Sistema na BeagleBone

Prof. Francisco Helder

22 de abril de 2026

Sumário

1 Apresentando o Hardware **2**
1.1 Visão Geral da Sequência de Boot 3
1.1.1 Mapeamento de Memória do Boot ROM 3
1.1.2 Loop Principal do Boot ROM 4
1.2 Initializaçao da Placa 4

2 Carregando a Primeira Aplicação Bare Metal **4**
2.1 Instalando o cross-compilador 5
2.1.1 Baixando o arm-none-eabi 5
2.2 O PC deve ser configurado com um servidor TFTP 7
2.3 Carregando uma Aplicação Bare Metal na BeagleBone 9

1 Apresentando o Hardware

O AM335X é um processador ARM Cortex-A8 da Texas Instruments, amplamente utilizado em sistemas embarcados, sendo a base da placa BeagleBone Black, com as principais características mostradas na Tabela 1. Esta prática detalha o processo de inicialização (boot) em modo bare-metal, ou seja, sem a presença de um sistema operacional.

Tabela 1: Principais componentes da arquitetura AM335x.

Componente	Descrição
CPU	ARM Cortex-A8 1 GHz
SRAM Interna	128 KB (64 KB para ROM, 64 KB para RAM)
Onboard Flash	4GB, 8bit Embedded MMC
Serial Port	UART0 access via 6 pin 3.3V TTL Header
Ethernet	10/100, RJ45
SD/MMC Connector	microSD , 3.3V
DDR Controller	Suporte a DDR2/DDR3 até 800 MHz
Periféricos	GPIO, UART, SPI, I2C, USB, Ethernet, etc.

A Figura 1 abaixo mostra a localização dos conectores, LEDs e interruptores no diagrama da placa de circuito impresso (PCB).

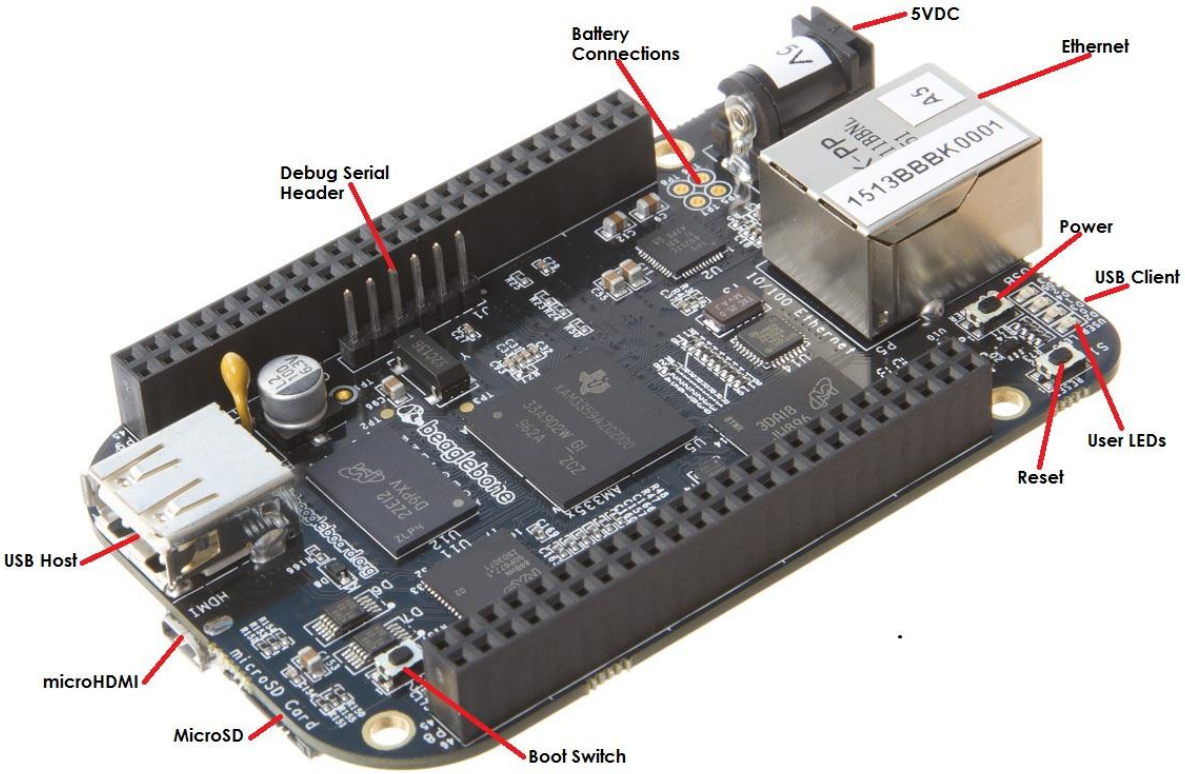

Figura 1: Placa BeagleBone com LEDS, conectores e componentes.

1.1 Visão Geral da Sequência de Boot

O processo começa no Power-On Reset (POR), onde a lógica de reset força o núcleo ARM Cortex-A8 a iniciar a execução a partir do Boot ROM interno de 176 KB. O Boot ROM funciona como o Initial Program Loader (IPL) ou Primary Program Loader (PPL).

O mapa de memória da BeagleBone Black é definido pelo processador TI Sitara AM335x (baseado no ARM Cortex-A8). Este SoC utiliza um espaço de endereçamento de 32 bits, totalizando 4 GB de memória endereçável. Na Figura 2 estão as regiões principais que você precisará para desenvolvimento bare-metal ou configuração de drivers:

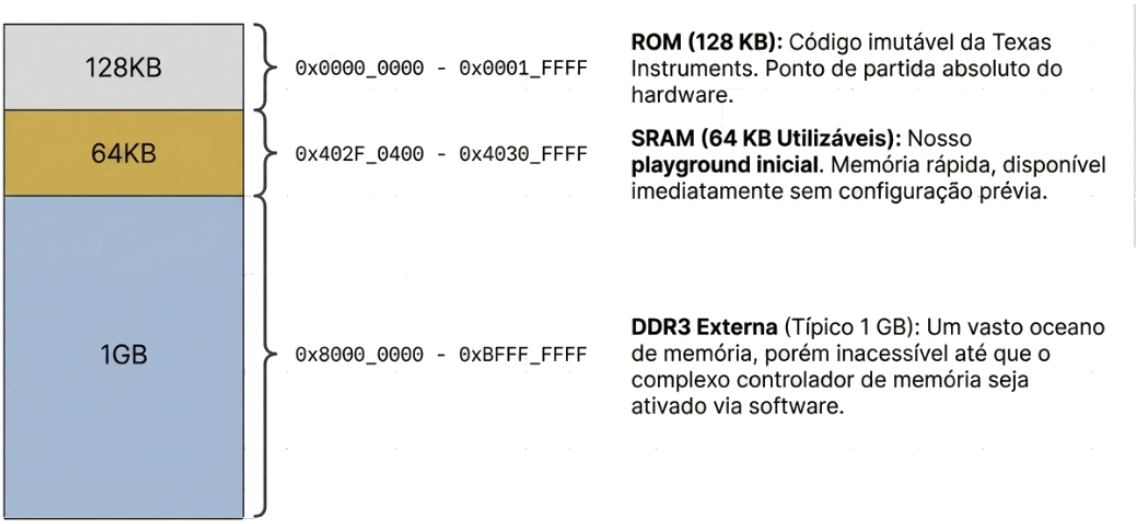

Figura 2: Mapa de memória da BeragleBone Black.

1.1.1 Mapeamento de Memória do Boot ROM

O Boot ROM está localizado fisicamente em `0x4000_0000`, mas é aliased (mapeado em espelho) para `0x0000_0000` no momento do reset. Ao iniciar o processador, o código de boot (ROM) carrega o primeiro estágio do bootloader (SPL ou MLO) na SRAM interna, e a sequência interna é:

- O processador inicia no mundo seguro a partir do endereço zero
- O ROM privado (secure) realiza a configuração inicial de segurança
- Em seguida, muda para o mundo público e salta para o Public ROM em `0x0002_0000` (alias de `0x4002_0000`)
- O vetor de reset no Public ROM aponta para o entry point em `0x0002_08d0`
- O endereço `0x402F_0400` é o ponto de entrada comum para códigos carregados via boot primário

Para configurar a multiplexação de pinos (Pinmuxing), que é essencial na placa, você utilizará o Control Module localizado em:

- **Base:** `0x44E1_0000`

Já para manipular os pinos de GPIOs diretamente em bare-metal os registradores são:

- **GPIO0:** `0x44E0_7000`
- **GPIO1:** `0x4804_C000`
- **GPIO2:** `0x481A_C000`
- **GPIO3:** `0x481A_E000`

Embora o processador suporte até 1 GB, a BeagleBone Black padrão possui 512 MB. Ela mapeia diretamente a partir do endereço `0x8000_0000`. Se você estiver escrevendo um linker script, este é o local onde o kernel Linux ou sua aplicação principal geralmente reside.

1.1.2 Loop Principal do Boot ROM

O código do ROM executa a seguinte sequência:

1. Identifica o tipo de dispositivo;
2. Constrói uma lista de dispositivos de boot;
3. Ler o `SYSBOOT` até encontrar uma imagem válida.

1.2 Initializaçao da Placa

O processador ARM possui diferentes "modos" (Usuário, Sistema, Interrupção, etc.). Para rodar o sistema operacional ou o código principal, geralmente queremos o Modo Supervisor (SVC), definido no código de inicialização em assembler, visto no `startup.s` abaixo.

```
_start:

mrs r0, cpsr          @Move o conteudo do CPSR para r0
bic r0, r0, #0x1F     @Limpa os 5 bits menos significativos
orr r0, r0, #0x13     @Define 0x13, que corresponde ao Modo SVC
msr cpsr, r0          @Escreve de volta no CPSR para aplicar

ldr sp, =0x4030CDFC   \@6kB public stack  TMR 26.1.3.2

bl _main

.loop: b .loop
```

2 Carregando a Primeira Aplicação Bare Metal

Nesta primeira prática de laboratório tem-se o objetivo de prepara o desenvolvedor a configurar o ambiente de desenvolvimento para programação em sistema embarcado, utilizando o bootloader U-Boot para carregar o primeiro sistema desenvolvido para a placa Bealgebone, definida pelo nome de bare metal, sendo carregada na placa via TFTP.

Em uma observação mais simples, a programação bare-metal significa escrever uma aplicação diretamente em seu hardware sem usar uma interface de programação de aplicativo externa, ou seja, sem nenhum sistema operacional. Escrevemos aplicações embarcadas acessando diretamente os registradores de hardware do mapa de memória dos microcontroladores.

2.1 Instalando o cross-compilador

Um cross-compilador nada mais é do que um compilador para uma plataforma diferente do computador usado no desenvolvimento. Como por exemplo um computador com a arquitetura x86 compilando para uma placa com plataforma ARM cortex-A8 de 32 bits. O cross-compilador que será usado será o arm-none-eabi, que é o cross-compilador específico para essa plataforma, e que tem característica de programação em bare metal.

2.1.1 Baixando o arm-none-eabi

O pacote com o cross-compilador **arm-none-eabi** pode ser encontrado no site próprio da ARM e algumas outras fontes como no site do Linaro. Acesse esse link https://www.linaro.org ou https://developer.arm.com e baixe a opção de cross-compilar bare metal que rode na máquina x86, algo parecido com (**gcc-arm-none-eabi-XX.X-XXXX.XX-x86_64-linux**). Após baixar o cross-compilador, será preciso criar uma pasta no diretório da disciplina, em que o desenvolvedor irá realizar suas práticas, nesse caso siga os seguintes passos no terminal:

```
$ mkdir lab
$ cd lab/
```

Após baixado o cross-compilador, será preciso criar um diretório nesse ambiente de desenvolvimento, nesse caso siga os seguintes passos no terminal:

```
$ mkdir toolchain
```

Após a criação do diretório é preciso descompactar o arquivo dentro de **toolchain**, digitando o seguinte comando:

```
$ tar jxvf gcc-arm-none-eabi-XX.X-XXX.X-x86_64-linux.tar.bz2 -C
   toolchain/
```

> Lembre-se de verificar o nome do arquivo antes de copiar e descompactar, pois obviamente se tiver um nome diferente, o linux não vai encontrar!!!

Com isso, já temos o ambiente de cross-compilação, porém para que ele seja acessado de qualquer parte do computador é preciso que seja mudada uma variável de ambiente chamada **$PATH**, para isso é preciso modificar o arquivo **.bashrc** que está no diretório do usuário, siga os seguintes passos para a modificação da variavel.

```
$ cd ~
$ gedit .bashrc
```

Em que a pasta "∼" é o diretório principal do usuário e **gedit** é um editor de textos (pode ser usado qualquer outro). Após abrir esse arquivo é preciso adicionar a seguinte linha no final do arquivo:

```
PATH=$PATH:~/lab/toolchain/gcc-arm-none-eabi/bin
```

Veja que **∼/lab/toolchain/gcc-arm-none-eabi/bin** é o caminho da pasta que foi criado, portanto adicione o caminho da sua pasta, que pode ser visto no terminal com o comando "pwd". Caso o caminho contenha espaços, como por exemplo **Área de Trabalho**, é necessario uma barra invertira(\), portanto, ficaria **Área\ de\ Trabalho**. Verifique seu caminho e cole corretamente no arquivo **∼bashrc**. Após configurar a variavel **$PATH**, o cross-compilador já pode ser usado normalmente, como visto na Figura 3.

```
heldercs@machine:lab$ arm-none-eabi-
arm-none-eabi-addr2line         arm-none-eabi-gcc-nm            arm-none-eabi-ld.bfd
arm-none-eabi-ar                arm-none-eabi-gcc-ranlib        arm-none-eabi-lto-dump
arm-none-eabi-as                arm-none-eabi-gcov              arm-none-eabi-nm
arm-none-eabi-c++               arm-none-eabi-gcov-dump         arm-none-eabi-objcopy
arm-none-eabi-c++filt           arm-none-eabi-gcov-tool         arm-none-eabi-objdump
arm-none-eabi-cpp               arm-none-eabi-gdb               arm-none-eabi-ranlib
arm-none-eabi-elfedit           arm-none-eabi-gdb-add-index     arm-none-eabi-readelf
arm-none-eabi-g++               arm-none-eabi-gdb-add-index-py  arm-none-eabi-size
arm-none-eabi-gcc               arm-none-eabi-gdb-py            arm-none-eabi-strings
arm-none-eabi-gcc-10.2.1        arm-none-eabi-gprof             arm-none-eabi-strip
arm-none-eabi-gcc-ar            arm-none-eabi-ld
heldercs@machine:lab$ arm-none-eabi-
```

Figura 3: Ambiente de cross-compilação instalado.

Após a instalação do ambiente de compilação, compile o os código `startup.s` e `main.c`, então tente rodar no computador. O computador não conseguirá rodar a aplicação, pois foi compilado para outra plataforma.

```c
#include "BeagleBone_regs.h"

static void delay(){
  volatile unsigned int ra;
  for(ra = 0; ra < 1000000; ra ++);
}

void putCh ( unsigned char c ){
  while(!(UART0_LSR & (1<<5)));
  UART0_THR = c;
}/* -----  end of function putCh  ----- */

unsigned int printS ( unsigned char *str, unsigned int length ){
  for(int i = 0; i < length; i++)
    putCh(str[i]);
  return(length);
}/* -----  end of function printS  ----- */

void _main ( ){
  const char *hello = "Hello World!!!\n\r";
  unsigned char *h = (unsigned char *)hello;
  while(1){
    printS(h,16);
    delay();
  }
}/* -----  end of function main  ----- */
```

Para compilar programas como esse, é preciso alguns parâmetros de compilação. Após a criação do arquivo, temos que adicionar esses parâmetros no makefile, usando o seguinte comando:

```makefile
CROSS_COMPILE = arm-none-eabi-
CC = $(CROSS_COMPILE)gcc
LD = $(CROSS_COMPILE)ld
OBJCOPY = $(CROSS_COMPILE)objcopy

CFLAGS = -mcpu=cortex-a8 -O2 -g -Wall -Iinc
```

```
LDFLAGS = -T src/memmap.ld

SRCS = src/startup.s \
    src/main.c

OBJS = $(addprefix obj/, $(notdir $(SRCS:.c=.o)))
OBJS := $(OBJS:.s=.o)

TARGET = bin/app_baremetal

all: $(TARGET).bin

obj/%.o: src/%.c
  $(CC) $(CFLAGS) -c $< -o $@

obj/%.o: src/%.s
  $(CC) $(CFLAGS) -c $< -o $@

$(TARGET).elf: $(OBJS)
  $(LD) $(LDFLAGS) $^ -o $@

$(TARGET).bin: $(TARGET).elf
  $(OBJCOPY) -O binary $< $@

clean:
rm -f obj/*.o bin/*.dump bin/*.elf bin/*.bin
```

Uma vez gerado o executavel, ao tentar executar, o computador não vai conseguir executar. Para saber para qual arquitetura o executavel tá compilado, simplesmente use o comando **file** no terminal, como mostrado no Figura 4.

```
heldercs@machine:pratica_01$ make
arm-none-eabi-gcc -mcpu=cortex-a8 -O2 -g -Wall -Iinc -c src/startup.s -o obj/startup.o
arm-none-eabi-gcc -mcpu=cortex-a8 -O2 -g -Wall -Iinc -c src/main.c -o obj/main.o
arm-none-eabi-ld -T src/memmap.ld obj/startup.o obj/main.o -o bin/app_baremetal.elf
arm-none-eabi-objcopy -O binary bin/app_baremetal.elf bin/app_baremetal.bin
heldercs@machine:pratica_01$ file bin/app_baremetal.elf
bin/app_baremetal.elf: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, with debug_info, not stripped
heldercs@machine:pratica_01$
```

Figura 4: Tentando rodar o executavel para o ARM.

2.2 O PC deve ser configurado com um servidor TFTP

O *Trivial File Transfer Protocol* (TFTP) fornece uma forma minimalista para transferir arquivos. É geralmente usado como uma parte da inicialização do PXE ou para atualizar configuração ou firmware em dispositivos que possuem memória limitada, tal como roteadores, telefones IP e sistema embarcado como um todo. Para Instalar o serviço de tftpd realize os seguintes passos:

1 instale os seguintes pacotes

```
$ sudo apt-get install tftpd-hpa tftp-hpa
```

2 Abra o arquivo `tftp-hpa` no caminho `/etc/default/tftpd-hpa`, e então edite o seguinte conteúdo

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/srv/tftpboot"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--secure"
```

3 Crie e configure o diretório `/srv/tftpboot`

```
$ sudo mkdir -p /srv/tftpboot
$ sudo chmod -R 777 /srv/tftpboot
$ sudo chown -R nobody /srv/tftpboot
```

4 Aplique a mudanças reiniciando o tftpd através do seguinte comando

```
$ sudo systemctl restart tftpd-hpa
$ sudo systemctl enable tftpd-hpa
```

5 Realize um teste de validação do seu serviço tftp. Crie um arquivo “hda.txt” no diretório (tftp) e transfira o arquivo hda.txt para o diretório qualquer (ex: Downloads).

```
$ cd Downloads
$ touch /srv/tftpboot/hda.txt
$ echo "somente um teste..." > /srv/tftpboot/hda.txt
$ chmod 777 /srv/tftpboot/hda.txt
$ ls -l /srv/tftpboot/
-rwxrwxrwx  1 hedercs heldercs 0 2010-08-31 15:34 hda.txt
$ tftp 127.0.0.1
tftp> get hda.txt
Sent 722 bytes in 0.0 seconds
tftp> quit
$ ls -l
-rwxrwxrwx  1 heldercs heldercs 707 2010-08-31 15:34 hda.txt
```

6 Copie a aplicação exemplo para o diretório TFTP:

```
$ cp bin/app_baremetal.bin /srv/tftpboot/
$ cd /srv/tftpboot
```

7 Crie o link simbólico

```
$ ln -s app_baremetal.bin download.bin
```

2.3 Carregando uma Aplicação Bare Metal na BeagleBone

Para carregar um sistema na BeagleBone é necessário seguir os seguintes passos:

- Reboot a placa e entre no U-boot apenas pressionando qualquer tecla. Você deve está conectado via cabo serial.
- Configure a variável de ambiente do boot para carregar uma imagem via tftp.

consfigure o seguinte script na CLI (*Command Line Interface*) do bootloader na BeagleBone:

```
U-Boot# setenv app "setenv autoload no;setenv ipaddr 10.4.1.2; setenv serverip 10.4.1.1; tftp 0x80000000 \
/tftpboot/download.bin;echo ***Booting to BareMetal ***;go 0x80000000"
```

em seguinda execute o script:

```
U-Boot# run app
```

...e temos o primeiro sistema rodando na placa BeagleBone, como visto na Figura 5...

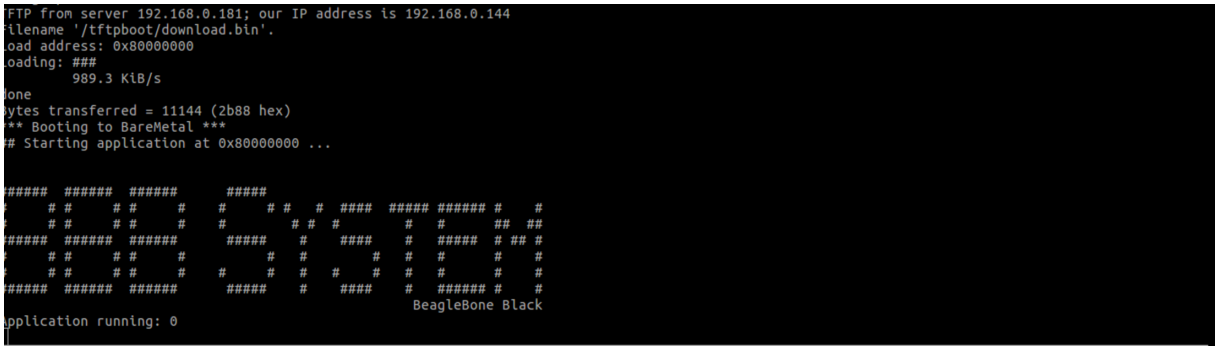

Figura 5: Primeiro sistema rodando na BeagleBone.